AUTHORITATIVE PROJECT SPECIFICATION

Product Vision, Modeling Scope, and Verification Architecture

Browser-based professional SysML modeling application

Governing source for product decisions, model architecture, implementation prompts, acceptance testing, workbook imports, and future production qualification.

Document field	Value
Status	Authoritative development baseline
Version	1.0
Date	August 7, 2026
Applies to	Professional browser-based SysML modeling application
Normative terms	Must = mandatory; should = recommended; may = permitted
Primary principle	Verify at the lowest technically valid level while preserving complete upward and lateral traceability

This specification defines required product behavior. It does not claim that planned or conceptual capabilities are already implemented.

Document Control and Capability Status

This document is the controlling product-level specification for the modeling and verification architecture. When a future implementation prompt, workbook schema, UI design, or acceptance test conflicts with this document, the conflict must be resolved explicitly through an architecture decision rather than silently implemented.

Status	Meaning	Permitted claim
Implemented	Capability is present and has repeatable evidence in the current baseline.	May be described as available only within the tested boundary.
Partial	Some workflow exists, but semantics, reliability, persistence, or coverage are incomplete.	Must identify limitations and must not imply qualification.
Planned	Approved requirement for a future stage.	May guide architecture and acceptance tests; must not be advertised as working.
Conceptual	Candidate capability or unresolved architecture direction.	Must remain non-binding until an architecture decision approves it.

Current application baseline

The known current baseline is a standalone/offline browser modeler that starts with a blank project and includes projects, diagrams, ports, properties, connectors, undo/redo, Excel and Groovy/model-script import, validation, traceability matrices, baselines, and JSON/CSV exchange. Each item must be requalified against acceptance evidence before it is marked Implemented in a release baseline. The application has no predefined FSBS or site data and does not read native CATIA/3DEXPERIENCE project files.

The professional semantic repository, type/instance architecture, configuration-bound evidence, explainable verification-scope engine, complete change-impact analysis, test-execution record, approvals, and multi-site rollup are Planned unless separately demonstrated.

Normative interpretation

- Must identifies a requirement whose absence prevents conformance to this product specification.
- Should identifies a recommended design that may be varied only with documented rationale.
- May identifies an optional capability that cannot weaken a must requirement.
- A visual diagram, matrix, dashboard, or report is always a view of repository facts and never an independent authoritative data source.

1. Product Purpose

The application must create, manage, analyze, and verify a complete, semantically correct SysML model for complex programs containing multiple sites, systems, subsystems, assemblies, components, replaceable units, interfaces, requirements, behaviors, configurations, Test Cases, executions, and evidence. It must function as a semantic systems-engineering repository and verification workbench, not merely as a diagram editor.

An engineer must be able to select a site, system, assembly, component, installed usage, or serial-numbered unit and determine exactly which requirements, verification methods, Test Cases, evidence, interfaces, dependencies, and higher-level effects apply. Unrelated program content must remain discoverable but excluded from the active verification boundary.

Product outcomes

- Author and navigate valid SysML 1.x models with professional notation and semantics.
- Maintain one semantic identity for an element that may appear on multiple compatible diagrams.
- Determine unit-level verification scope without losing interface, dependency, configuration, or higher-level context.
- Plan, record, review, and approve verification using evidence bound to a specific execution and configuration.
- Support reusable definitions across many sites while preserving site-installed identity, local deviations, and local evidence.
- Import and reimport structured workbooks transactionally without silent duplication, orphaning, or provenance loss.

2. Governing Verification Principle

Verification must occur at the lowest technically valid level of decomposition while preserving traceability to higher-level requirements, interfaces, configurations, dependencies, and program objectives.

The correct boundary is the smallest set of semantic elements and conditions sufficient to make a defensible verification decision. The scope engine must begin with the selected element, instance, configuration, baseline, and verification objective; traverse only relationship types authorized by explicit rules; evaluate applicability and evidence identity; and classify every candidate as direct, interface, dependency, regression, higher-level, or out of scope.

Boundary determination algorithm

1. Resolve the selected semantic element and, where verification evidence is involved, require an instance or usage, configuration identity, and baseline.
2. Collect applicable requirements through containment/decomposition, allocation, satisfy, verify, refine, and approved trace rules without treating those relationships as equivalent.
3. Classify requirements verifiable at the selected level as direct; retain those requiring emergent behavior at the nearest valid higher level.
4. Traverse modeled ports, connectors, Item Flows, interface constraints, shared resources, and behavioral dependencies only to the depth justified by the verification objective.
5. Evaluate changes and suspect links to add regression candidates.
6. Apply explicit exclusions for unrelated branches; record a machine-readable reason for every inclusion and exclusion.
7. Calculate coverage using only applicable requirements and admissible evidence. Present missing model facts as unresolved problems, not assumptions.

The application must prevent excessive scope by limiting traversal to semantically relevant paths, and insufficient scope by warning when interfaces, configurations, allocations, or required relationships are missing or ambiguous.

3. Verification-Scope Categories

Category	Inclusion rule	Exclusion rule	Traceability and user explanation	Warnings	Reports	Example
Direct	Requirement is applicable, allocated to/satisfied by the selection, and verifiable at this level.	Exclude if only a coincidental trace or if verification requires emergent behavior.	Show relationship path, applicability, method, evidence state, and boundary rationale.	Missing allocation, method, Test Case, evidence, or configuration identity.	Unit scope report; direct coverage; requirement-to-evidence view.	Tire pressure-retention requirement verified on the installed tire configuration.
Interface	Requirement constrains an interaction across a modeled boundary used by the selection.	Exclude disconnected interfaces and exchanges irrelevant to the objective.	Show both endpoints, interface definition, connector path, exchanged item, constraint, and test representation.	Unresolved endpoint, missing Item Flow, incompatible interface, or unqualified simulator.	Interface scope; interface coverage; test-configuration adequacy.	Wheel/tire mounting interface and vehicle-load boundary conditions.
Dependency	Selection depends on a shared service, resource, function, constraint, behavior, or infrastructure needed for the objective.	Exclude mere package proximity or unrelated common services.	Show dependency type, provider, consumer, requirement path, and necessity.	Circular/implicit dependency or missing provider configuration.	Dependency map; supporting-evidence report.	Inflation service and calibrated pressure measurement equipment.
Regression	Prior result may be suspect because a relevant element, relationship, procedure, environment, or configuration changed.	Exclude changes proven semantically isolated by impact rules and evidence.	Show changed fact, traversed paths, affected result, severity, and retest rationale.	Unknown baseline delta or incomplete change provenance.	Impact report; suspect-link view; regression plan.	Tire material revision makes durability evidence suspect but not unrelated engine evidence.
Higher-level	Requirement traces to the selection but can be concluded only at assembly, system, site, or program level.	Exclude if a complete, approved lower-level method provides the required conclusion.	Show nearest valid verification level and the contribution expected from the selection.	Attempt to grant full lower-level credit for emergent performance.	Rollup report; contribution matrix; deferred verification list.	Vehicle handling performance requires vehicle-level testing using tire evidence as an input.
Out of scope	No authorized semantic path connects the element to the selected objective.	Do not exclude a relevant interface, dependency, or higher-level contribution merely because it is outside containment.	Show the failed/absent relevance rule and allow drill-through without adding it to active scope.	Unmodeled relationship may cause false exclusion.	Exclusion ledger; unresolved-model warning report.	Engine emissions are excluded from tire pressure verification.

4. Program and Model Hierarchy

The repository must support Program → Site → System → Subsystem → Assembly → Component → Unit as a common decomposition pattern, but it must not enforce it as an inflexible containment template. Metamodel rules must permit alternate valid structures while preserving explicit ownership, decomposition kind, type/instance identity, and verification applicability.

Concern	Authoritative meaning	Must remain distinct from
Semantic ownership	Repository namespace and lifecycle owner of an element.	Diagram placement, responsibility, allocation, and decomposition.
Functional decomposition	Breakdown of behavior or function into subordinate behaviors.	Physical parts and package ownership.
Physical decomposition	Whole-part structure of physical usages/components.	Type generalization and organizational ownership.

Concern	Authoritative meaning	Must remain distinct from
Logical decomposition	Technology-independent logical responsibilities and interfaces.	Installed physical configuration.
Organizational responsibility	Person, team, role, or organization accountable for work or approval.	Semantic owner and physical owner.
Requirement allocation	Assignment of responsibility or applicability to model elements.	Satisfaction, verification, and containment.
Verification scope	Objective-specific computed boundary.	Containment subtree or current diagram contents.
Diagram presentation	View-specific nodes, edges, labels, layout, and compartments.	Semantic identity and ownership.

A diagram must have an explicit diagram owner package and a separate semantic context. The same semantic element may appear on multiple compatible diagrams through distinct presentation objects. Deleting a presentation must not delete its semantic element unless the user performs a separately confirmed semantic deletion. Child diagrams must be created only where decomposition is actually modeled.

5. Type and Instance Architecture

The repository must distinguish reusable definitions from usages and real-world instances. A Block or Interface Block defines reusable structure or contract; a part property or usage applies that type in a context; an instance represents a specific occurrence; a serial-numbered unit is an instance with controlled identity and lifecycle records.

Concept	Required identity and behavior	Verification consequence
Reusable type	Stable ID, version, owner, definition, features, ports, constraints.	Common requirements and Test Cases may be authored against the type but cannot alone prove every instance.
Site-specific usage	Owning context, role name, multiplicity, type, local redefinition, applicability.	Determines installed context and local interfaces.
Serial-numbered unit	Instance ID, serial/asset ID, type revision, manufacture/maintenance history.	Every execution must bind to this identity or a justified equivalence class.
Variant/alternative	Explicit variation point, selected option, compatibility rules.	Evidence is valid only for covered selections.
Revision	Immutable or controlled version identity and change set.	Changes trigger impact and suspect evaluation.
Baseline	Approved set of versions and relationships at a time.	Scopes and evidence conclusions must cite the baseline.
Configuration state	As-designed, as-built, as-installed, as-tested, or as-maintained snapshot.	Credit requires compatibility between tested and claimed states.

Verification-result reuse

A result may be reused only through an explicit Verification Credit Reuse record that identifies the source execution, target unit/configuration, equivalence basis, compared characteristics, allowed differences, analytical justification, approving authority, validity period, and any residual risk. Similarity must not be inferred from a shared type name alone.

- Required evidence includes controlled configuration comparisons, interface equivalence, operating-range coverage, procedure and equipment suitability, environment representativeness, unresolved deviation assessment, and approval.
- Any later change to an equivalence-critical characteristic must mark the reuse record and dependent verification conclusions suspect.
- Reuse may grant full, partial, or no credit; partial credit must identify remaining verification obligations.

6. Requirement Architecture

Field group	Normative requirement
Stable internal ID	Immutable repository identity; never user-repurposed.
External ID	Stable interchange identity and reimport key within a declared source namespace.
Requirement ID	Human-facing controlled identifier; uniqueness enforced in scope.
Name and text	Name summarizes; text contains the complete normative statement.
Source and owner	Origin/provenance and accountable semantic owner.
Type/category	Requirement class used by rules, views, and governance.
Rationale	Reason and context; never substitutes for normative text.
Verification method	Approved method or method candidates with rationale.
Risk/criticality	Drives assurance, approval, and evidence rigor.
Status/version/baseline	Controlled lifecycle and configuration identity.
Applicability	Expression over sites, types, instances, variants, configurations, and operating conditions.
Allocated/satisfying elements	Explicit responsibilities and design claims.
Verifying Test Cases/evidence	Plan-level relationship plus execution-derived evidence chain.
Change history/suspect	Auditable revisions and impact state.

Relationship	Correct meaning/direction	Scope contribution
Containment	Owner contains owned requirement; direction owner → owned.	Namespace and lifecycle only; not automatic applicability.
Requirement decomposition	Parent requirement is decomposed into child obligations.	Children contribute when decomposition is complete and applicability holds; parent conclusion follows declared rollup rules.

Relationship	Correct meaning/direction	Scope contribution
deriveReqt	Derived requirement depends on source requirement rationale.	Trace/impact path; not automatic allocation or verification.
satisfy	Design element → requirement; asserts design satisfaction responsibility.	Supports direct candidates; does not prove verification.
verify	Test Case → requirement; states intended verification coverage.	Selects planned verification; link alone never grants status.
refine	Model element → requirement; adds precision or model expression.	May identify relevant behavior/constraint but does not grant credit.
trace	Weak, explicitly characterized dependency.	Traceability only unless a governed rule authorizes traversal.
copy	Copied requirement → master requirement with controlled synchronization semantics.	Applicability depends on copy policy; not evidence inheritance.
allocate	Source → target assignment of responsibility or mapping.	Contributes only according to allocation kind and applicability.

7. Verification Architecture

Semantic role	Required properties and behavior
Verification Requirement	Controlled verification obligation or strategy derived from one or more product requirements.
Verification Method	Enumerated method plus method-specific inputs, evidence, level, approval, limitations, and reuse rules.
Test Case	Reusable semantic definition of objective, preconditions, inputs, expected outcomes, covered requirements, and method.
Test Procedure	Controlled ordered procedure revision that realizes a Test Case.
Test Step	Atomic action/observation with expected result, data capture, and acceptance rule.
Verification Plan	Planned collection of verification activities for a scope, baseline, schedule, responsibility, and approval.
Planned Test	A plan-specific use of a Test Case with assigned unit/configuration, procedure, environment, and schedule.
Test Configuration	Controlled assembly of test article, interfaces, simulators, software, equipment, and settings.
Test Environment/Equipment	Qualified environmental conditions and uniquely identified/calibrated resources.
Test Execution	Immutable occurrence of a planned or authorized test with actual identities, timestamps, participants, and deviations.
Test Result	Outcome record for execution, step, measurement, and acceptance rule.
Measurement	Value, unit, uncertainty, timestamp, source channel, and calibration trace.
Expected/Actual Result	Controlled expectation and observed outcome; differences are evaluated.
Determination	Pass, fail, conditional, inconclusive, not run, blocked, or waived with authority.
Anomaly/Corrective Action/Retest	Controlled discrepancy lifecycle linked to affected results, requirements, changes, and follow-on execution.
Waiver/Deviation	Approved departure with scope, rationale, risk, authority, conditions, and expiration.

Semantic role	Required properties and behavior
Verification Evidence	Immutable or controlled artifact reference, hash, provenance, classification, and retention.
Verification Report/Approval	Generated conclusion package and signed/attributed decision record.

A Test Case is reusable. A Planned Test is one intended use. A Test Execution is one historical occurrence. Evidence is the preserved record produced or referenced by that occurrence. Requirement verification status is a governed conclusion derived from admissible evidence—not a property manually toggled because a verify link exists.

8. Verification Methods

Method	Required inputs	Expected evidence	Level	Approval	Limitations	Reuse conditions
Test	Controlled article/configuration, procedure, equipment, environment, acceptance criteria.	Execution record, measurements, results, evidence.	Any level with observable response.	Authorized execution and result approval.	May be costly/destructive; representativeness mandatory.	Equivalent unit/configuration and procedure/environment coverage.
Analysis	Inputs, assumptions, model/tool/version, equations, uncertainty, acceptance criteria.	Reproducible calculation/model output and review.	Component through program.	Independent technical review for critical claims.	Validity bounded by assumptions and input quality.	Inputs, model version, domain, and uncertainty remain applicable.
Inspection	Controlled characteristics, method, instruments if applicable, acceptance criteria.	Inspection record, images/data, inspector identity.	Item through system.	Qualified inspector per governance.	Confirms observable attributes, not unobserved performance.	Same controlled characteristic and applicable sampling rule.
Demonstration	Scenario, operators, conditions, observable success criteria.	Witnessed performance record and evidence.	Assembly through site/program.	Authorized witness/approver.	Usually qualitative or operational; limited precision.	Scenario and configuration equivalence demonstrated.
Simulation	Validated model, inputs, boundary conditions, solver/tool version, uncertainty.	Reproducible runs, validation evidence, results.	Component through program.	Model credibility approval appropriate to criticality.	Cannot exceed validated domain.	Same domain, model credibility, and configuration mapping.
Certification	Applicable authority, standard, scope, configuration, validity.	Certificate and supporting controlled records.	Product/type/installation as defined by authority.	Authorized certification body.	Credit limited to certified scope and dates.	Target lies within explicit certificate applicability.
Similarity/equivalence	Source evidence, target comparison, critical characteristics, differences, rationale.	Approved equivalence assessment and residual-risk record.	Type/instance/configuration.	Designated verification authority.	Never inferred automatically.	Only through explicit approved reuse record.

9. Verification-Scope Engine

Inputs

- Selected semantic element and, when applicable, installed usage or serial-numbered instance.
- Selected configuration state and baseline.
- Requirement lifecycle status and applicability date.
- Verification method filter.

- Direct-only mode and independent toggles for interfaces, dependencies, regression impact, higher-level traceability, and previously verified requirements.
- Traversal depth and authorized rule set controlled by governance, not arbitrary graph expansion.

Outputs and explainability

- Included and excluded requirements, each with a category, relationship path, applicability evaluation, and plain-language reason.
- Required Test Cases; existing admissible results; missing, stale, suspect, conditional, and reused evidence.
- Interface and dependency paths; higher-level obligations; explicitly excluded systems.
- Coverage percentages with numerator, denominator, exclusions, waived items, and evidence-admissibility rules visible.
- Unresolved modeling problems that reduce confidence, including missing ownership, ambiguous allocation, unresolved endpoints, missing configuration, and incomplete requirement text.

The engine must persist a Scope Definition and reproducible Scope Evaluation record containing input identities, rule-set version, repository baseline, timestamp, results, and explanation graph. Re-running the same inputs against the same baseline and rule set must produce the same result.

Coverage

Default requirement coverage = requirements with an approved verification conclusion supported by admissible evidence ÷ applicable requirements in the selected scope. Planned, executed-but-unapproved, waived, inconclusive, and suspect items must be reported separately. A user may select alternate governed metrics, but the interface must show the formula and must not blend statuses invisibly.

10. Change-Impact and Regression Analysis

Changed fact	Candidate impact paths	Required analysis
Requirement text	Requirement and derived/decomposed children; Test Cases; conclusions.	Reassess meaning and verification adequacy.
Allocation	Old/new allocated elements, interfaces, plans.	Recompute direct and higher-level scope.
Component design	Satisfied requirements, interfaces, behaviors, constraints, evidence.	Classify changed characteristics; compare evidence applicability.
Port/connector/Item Flow/interface	Both endpoints, exchanged items, interface requirements, test configuration.	Recompute interface scope and simulator adequacy.
Value property/constraint	Dependent equations, budgets, analyses, results.	Re-evaluate affected analyses/simulations and limits.
Behavior	Refined requirements, scenarios, states, interactions, Test Cases.	Recompute behavioral coverage and regression candidates.
Test Case/procedure	Covered requirements, planned tests, executions using revision.	Preserve historical validity; assess future and claimed reuse.
Equipment/environment	Executions and measurements using identity/conditions.	Check calibration, qualification, and representativeness.

Changed fact	Candidate impact paths	Required analysis
Configuration	Evidence and scope bound to changed selections.	Compare critical characteristics and require requalification/reuse approval.
Model relationship	All traversals whose rationale uses the relationship.	Re-run affected scopes; do not invalidate unrelated branches.

Impact results must distinguish directly affected, indirectly affected, suspect, potentially invalid, required regression, and demonstrably unaffected items. Every result must include traversed relationship paths and rules. A local change must not invalidate every program requirement.

11. Interface-Aware Verification

The model must represent Interface Blocks, Proxy Ports, Full Ports, Flow Properties, Connectors, Item Flows, nested connector paths, provided/required features, interface constraints, exchanged signals/data/material/energy/control, environmental interfaces, and shared-resource dependencies. Verification scope must traverse these semantics rather than infer interfaces from visual line contact.

Representative or simulated interfaces

A unit may be tested against a simulator, emulator, fixture, load bank, environmental chamber, representative mating component, or analytical boundary condition only when the Test Configuration identifies the substitute and a Test Configuration Adequacy Assessment demonstrates equivalence for every verification-critical characteristic.

- The assessment must map each operational interface feature to the test representation.
- It must cover ranges, timing, protocols, impedance/load, tolerances, environmental conditions, faults, and uncertainty as applicable.
- Differences, limitations, residual risk, calibration/qualification, and approving authority must be recorded.
- A change to the operational interface or simulator must trigger suspect evaluation of dependent evidence.

12. Behavioral and Parametric Verification

Model concept	Verification connection
Activities/functions	Requirements refine or allocate to actions; object/control flows define scenarios, inputs, outputs, and observation points.
State machines	State invariants, transitions, events, guards, and timing define expected behavior and test coverage.
Interactions/sequences	Lifelines, messages, ordering, constraints, and alternatives define protocol and timing evidence needs.
Use cases	Stakeholder goal and scenario context organize requirements and validation; use-case presence alone is not verification.
Constraints/value properties/units	Quantitative requirements must bind to typed values, units, equations, ranges, tolerances, and uncertainty.

Model concept	Verification connection
Performance budgets	Contributions and rollup rules define allocation and higher-level verification.
Parametric analyses/simulations	Each run must capture model version, solver/tool, inputs, assumptions, configuration, outputs, and credibility evidence.

A behavioral or performance requirement must trace to the behavior, constraint, value, or scenario that makes its acceptance criteria measurable. Unit compatibility, dimensional consistency, range coverage, and assumptions must be validated before analysis evidence is admissible.

13. Multi-Site Modeling

Common libraries must contain reusable definitions—types, interfaces, ValueTypes, units, generic requirements, verification methods, and Test Cases. Site models must contain site identity, installed usages/instances, local configurations, local interfaces, site-specific requirements, executions, evidence, deviations, and approvals. A reusable definition and a site-installed occurrence must never be collapsed into one object.

Concern	Program-wide reusable layer	Site-specific layer
Structure	System/component/interface types.	Installed usages, serial units, local connections.
Requirements	Common master requirements and applicability rules.	Local requirements, copies only when governed, deviations.
Tests	Reusable Test Cases and procedure families.	Planned Tests, Test Configurations, executions, evidence.
Configuration	Variant rules and compatibility constraints.	As-designed/built/installed/tested/maintained snapshots.
Coverage	Normalized rollup definition.	Local status and admissible evidence.
Comparison	Shared characteristics and metrics.	Configuration deltas, anomalies, deviations, trends.

Program-wide coverage must aggregate site conclusions without erasing denominators, exclusions, waivers, reused credit, configuration differences, or local suspect status. Cross-site comparison must compare like-for-like configurations or show the mismatch.

14. Verification Views and Workbenches

View/workbench	Required behavior
Requirement tables/hierarchies	Filterable requirements with decomposition, applicability, ownership, status, and completeness.
Relationship/allocation/satisfy/verification matrices	Repository-derived cross-tab views with directional semantics and controlled editing.
Coverage dashboards	Scope-aware totals separated by approved, planned, missing, suspect, waived, reused, and not applicable.

View/workbench	Required behavior
Unit verification workspace	Selected instance/configuration/baseline, categorized scope, Test Cases, evidence, anomalies, and actions.
Test-plan views	Planned Test schedule, responsibility, readiness, configuration, method, and approval.
Execution history	Immutable chronological executions, revisions, results, evidence, anomalies, and retests.
Configuration/baseline comparison	Semantic deltas with evidence and applicability impact.
Change-impact/suspect-link views	Explainable paths from change to affected requirements, tests, evidence, and regression actions.
Evidence review/approval	Evidence preview/reference, provenance, completeness checks, reviewer decision, signature/identity, and audit.
Reports	Reproducible generated scope, coverage, verification, impact, and audit reports with baseline and query definition.

Every view must query the semantic repository. A view may store layout, filters, saved queries, and user annotations, but must not maintain independent copies of engineering facts.

15. Model Governance

- Stable internal IDs must be immutable; External IDs must be unique within a declared source namespace; qualified names must be deterministically derived from complete owner chains.
- Naming conventions, allowed metaclasses/stereotypes, relationship direction, ownership, and required fields must be machine-validatable.
- Versions and baselines must preserve historical identity and allow reproducible scope/evidence conclusions.
- Change control must record author, timestamp, reason, before/after values, affected baseline, review, and approval.
- Roles and permissions must separate authoring, executing, reviewing, approving, administration, and verification authority.
- Audit history must be append-only or tamper-evident at the product's chosen assurance level.
- Data and import provenance must identify source, file/version/hash where available, importer/rule version, timestamp, operator, and reconciliation outcome.
- Evidence retention rules must prevent deletion or mutation that would invalidate approved claims without an explicit supersession/audit trail.

Prevention of misleading claims

The application must block or clearly qualify a verified conclusion when requirement text is incomplete; applicability is unresolved; configuration identity is missing; evidence is absent, suspect, stale, or outside its validity; required approval is missing; the Test Case does not cover the acceptance criteria; or unresolved model errors affect the conclusion. Status labels must distinguish verified, conditionally verified, partially verified, waived, failed, inconclusive, not verified, and suspect.

16. Import and Interchange Requirements

Import concern	Normative behavior
Identity	Stable External IDs plus source namespace; qualified names used for diagnostics and controlled fallback, not as the sole identity when IDs exist.
Ownership	Every imported element must resolve a complete owner chain. No silent root-level or unowned elements.
Diagrams	Explicit diagram owner and separate semantic context; presentation identities separate from semantic identities.
Requirements	Complete text, Requirement ID, owner, provenance, applicability, and relationship resolution.
Relationships	Source/target resolved by stable identity; direction and compatible types validated.
Type/instance	Explicit definitions, usages, instances, serial IDs, and configurations; no inference from naming alone.
Verification data	Test Cases, Planned Tests, executions, results, evidence references, configuration identity, and provenance imported as distinct records.
Dry run	Parse, validate, resolve, classify create/update/conflict/no-op, and preview changes without mutation.
Transaction	All-or-nothing commit by import unit; rollback on failure; durable import log.
Reimport	Preserve identity, detect duplicates, update allowed fields, protect controlled fields, reconcile deletions/renames/conflicts.
Validation	Reject or quarantine unresolved owners, endpoints, duplicate IDs, illegal relationships, missing required fields, and invalid diagram ownership.

Child diagrams must be imported or created only where decomposition is semantically modeled. Reimport must never multiply presentations, owners, packages, or semantic elements merely because names changed.

17. SysML Scope

Concept	Classification	Purpose/qualification
Package, Model, Namespace, Element, Comment	Essential baseline	Repository organization and documentation.
Block, ValueType, DataType, Enumeration, Unit, QuantityKind	Essential; unit verification; multi-site	Structural and quantitative foundation.
Part/reference/value properties, multiplicity, generalization	Essential; unit verification	Composition, usages, reusable typing.
Interface Block, Proxy/Full Port, Flow Property, Connector, Item Flow	Essential; unit verification; multi-site	Interface-aware scope and test configuration.
Requirement and deriveReqt/satisfy/verify/refine/trace/copy	Essential; unit verification	Requirement architecture and evidence chain.
Activity, action, object/control flow, parameter, partition	Essential; unit verification	Behavioral requirements and test procedures.

Concept	Classification	Purpose/qualification
State machine, state, transition, event, guard	Essential; unit verification	Mode-dependent verification.
Interaction, lifeline, message, combined fragment	Essential; unit verification	Protocol and timing verification.
Use case, actor, include, extend, generalization	Essential	Stakeholder goals and scenario context.
Constraint Block, constraint property, binding connector, parametric diagram	Essential; unit verification	Analysis and performance verification.
Instance Specification, slots	Essential; unit verification; multi-site	Instance/configuration representation.
Allocation and dependency variants	Essential; multi-site	Responsibility, mapping, and dependency semantics.
Profiles/stereotypes/tagged values	Essential	Controlled domain extension.
Ports/services beyond selected SysML profile, advanced UML deployment	Required later	Adopt only with explicit use cases and tests.
SysML 2.0 language/API conformance	Out of current scope	No conformance claim until separately implemented and qualified.

Formal notation requirements govern symbols, compartments, labels, relationship lines/arrowheads, diagram compatibility, and semantic creation. Workflow features—drag/drop, property editing, matrices, dashboards, imports, collaboration, scope generation, and reports—must preserve those semantics but are not themselves SysML notation.

18. End-to-End User Journey: Replaceable Power-Supply Unit

8. The engineer opens a multi-site communications program and selects the Cutler site model.
9. In the installed system structure, the engineer selects powerSupplyUnit SN-042 rather than the reusable power-supply type.
10. The engineer selects the as-installed configuration baseline B-17 and a verification objective.
11. The scope engine validates identity and generates a reproducible scope evaluation.
12. The workspace separates direct requirements from connector, power-input, cooling, control, and environmental interface requirements.
13. Unrelated antenna and transmitter-subsystem requirements appear in the exclusion ledger, not the active scope.
14. Missing Test Cases and requirements with incomplete acceptance criteria are flagged.
15. The engineer assigns reusable Test Cases or creates new controlled Test Cases linked through verify relationships.
16. A Test Configuration binds SN-042, its software/firmware revision, load bank, interface simulator, calibrated equipment, procedure revision, and environmental conditions.
17. The engineer records a Test Execution with actual identities, timestamps, participants, deviations, and step results.
18. Evidence files/references, measurements, hashes/provenance, and calibration references are attached.
19. The system evaluates expected versus actual results and proposes—but does not silently approve—determinations.

20. An authorized reviewer grants requirement status only where evidence is complete and applicable.
21. A voltage transient anomaly is linked to affected steps, results, requirements, and corrective action.
22. A connector design change is committed against a new baseline.
23. Impact analysis marks interface evidence and related verification conclusions suspect while showing unaffected mechanical inspection evidence.
24. The regression scope requires only the affected transient and interface tests plus justified dependencies.
25. Retest evidence supersedes or supplements the earlier execution without deleting history.
26. A unit verification report states configuration, baseline, scope rules, included/excluded requirements, evidence, anomalies, waivers, approvals, and coverage.
27. Status rolls upward only through defined decomposition/allocation rules; higher-level site availability remains pending until emergent system behavior is verified.

19. Non-Goals and Safeguards

- The application must not function only as a diagram-drawing tool.
- It must not duplicate semantic elements for every diagram appearance or treat diagram containment as semantic ownership.
- It must not include the entire program when a smaller technically valid scope is sufficient, nor ignore relevant interfaces or dependencies.
- It must not transfer test credit to untested configurations without explicit approved justification.
- It must not auto-create unnecessary child diagrams.
- It must not claim verification without admissible evidence or mark a requirement verified solely because a link exists.
- It must not silently lose provenance, identity, ownership, or relationship resolution during import.
- It must not claim CATIA, Cameo, SysML, SysML 2.0, or interchange certification beyond separately documented qualification evidence.
- It must not copy proprietary CATIA or Cameo source code, graphics, branding, icons, or assets. It may implement standards-based concepts and original professional interaction patterns.

20. Acceptance Criteria

Each acceptance record below is mandatory for the corresponding capability. “Automated test” means a repeatable test incorporated into the project test suite. A manual visual check may supplement but must not replace semantic assertions.

AC-01 Unit scope selection

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Program contains type, installed unit, configuration, baseline, allocations.	Select unit/configuration/baseline; generate scope.	Saved Scope Evaluation references exact identities and rule version.	Workspace identifies selected unit and categorized counts.	Reload reproduces same inputs/result.	Fixture asserts IDs, categories, deterministic result.	Type substituted for instance, missing config accepted, or non-determinism.

AC-02 Requirement allocation

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Applicable requirement and valid allocate/satisfy paths exist.	Inspect/edit allocation.	Direction, allocation kind, and applicability persist.	Matrices and property view agree.	Save/reload preserves one relationship.	Round-trip semantic graph assertion.	Duplicate/reversed relationship or inconsistent views.

AC-03 Interface expansion

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Selected unit has modeled ports/connectors/flows and interface requirements.	Enable interfaces.	Only relevant interface requirements and endpoints added.	Explanation shows connector path.	Scope record retains expansion choice.	Positive/negative graph traversal tests.	Missing relevant path or unrelated interface included.

AC-04 Out-of-scope exclusion

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Program contains unrelated subsystem.	Generate focused scope.	Unrelated requirements excluded with reason.	Exclusion ledger is visible and drillable.	Exclusions persist in evaluation.	Fixture checks exclusion and reason code.	Unrelated item counted in denominator or silently omitted.

AC-05 Configuration evidence

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Two units/configurations share a type; evidence exists for one.	View status for both.	Credit remains bound to tested identity unless approved reuse exists.	UI shows source and reuse state.	Bindings survive reload/export/import.	Cross-instance credit isolation test.	Automatic credit transfer.

AC-06 Coverage calculation

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Mixed approved, missing, suspect, waived, and N/A statuses.	Open coverage dashboard.	Numerator/denominator follow governed formula.	Counts, percentages, and formula visible.	Saved query reproduces value.	Parameterized status calculation tests.	Hidden blending or incorrect denominator.

AC-07 Suspect detection

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Approved evidence depends on changed requirement/interface/procedure.	Commit change.	Dependent conclusions become suspect with cause path.	Suspect view lists reason and changed baseline.	History persists before/after state.	Impact fixture asserts precise affected set.	No suspect flag or unrelated mass invalidation.

AC-08 Regression scope

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Change affects one branch plus one shared dependency.	Generate regression scope.	Affected direct/interface/dependency tests included; unrelated tests excluded.	Reasons shown for every item.	Evaluation is versioned.	Graph impact test with unaffected controls.	Whole-program regression or missed dependency.

AC-09 Requirement-to-evidence

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Approved execution and evidence cover a requirement.	Trace from requirement to evidence and back.	Chain includes Test Case, plan/use, execution, result, config, evidence, approval.	Bidirectional navigation shows identities.	Chain survives reload and export/import.	Referential-integrity traversal test.	Broken/missing hop or verify-link-only status.

AC-10 Multi-site reuse

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Shared type/Test Case used at two sites with distinct installations.	Create site plans/executions.	Definition reused once; executions/evidence remain site-specific.	Views show definition versus occurrences.	No duplication after reload/reimport.	Identity/count assertions.	Merged evidence or duplicated common definition.

AC-11 Result reuse control

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Source evidence and target config differ.	Request similarity credit.	No credit until assessment and approval; partial/full scope explicit.	Comparison and approval state visible.	Reuse record is auditable.	Permission/workflow and change-suspect tests.	Unapproved automatic credit.

AC-12 Save/reload integrity

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Representative semantic and presentation model exists.	Save, close, reopen.	All IDs, ownership, links, configurations, evidence refs, and presentations unchanged.	Same diagrams/views open correctly.	Multiple cycles remain stable.	Canonical serialization comparison.	Loss, duplication, or ownership drift.

AC-13 Import/reimport identity

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Workbook has stable IDs, complete owners, diagrams, relationships.	Dry run, import, modify source, reimport.	Creates/updates/no-ops/conflicts correctly; no orphans/duplicates.	Preview and reconciliation report match commit.	Transaction log and rollback retained.	Golden workbook idempotency/rollback tests.	Silent orphan, duplicate, partial commit, or ID change.

AC-14 Collaboration consistency

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Two authorized clients open same branch/model.	Perform concurrent semantic and presentation edits.	Operations converge or create explicit conflict; audit identities preserved.	Both clients show consistent state/conflict.	Reload matches authoritative revision.	Multi-client operation/convergence test.	Silent overwrite, project replacement, or divergent state.

AC-15 Auditability and reports

Preconditions	User action	Expected semantic result	Expected interface result	Persistence	Required automated test	Failure condition
Controlled model, changes, execution, approval exist.	Generate verification report and inspect audit.	Report cites baseline, scope definition, config, evidence, status, exclusions, approvals.	Readable report with drillable/source references where supported.	Regeneration from same baseline is equivalent.	Snapshot/golden assertions and audit-chain checks.	Untraceable claim, missing provenance, or mutable history.

21. Implementation Roadmap

1. Professional baseline and conformance qualification

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Establish stable editing, notation, persistence, import, validation, and test harness.	Current modular editor baseline and authoritative metamodel/notation rules.	Qualified element/diagram matrix; save/reload suite; import golden files; regression harness.	All baseline acceptance tests pass on supported browsers.	Do not add scope-engine conclusions or production approval claims.

2. Semantic hierarchy and type/instance completion

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Complete ownership, decomposition kinds, reusable types, usages, instances, and configuration identities.	Stage 1 repository integrity.	Type/instance metamodel, UI, migration, validators, configuration snapshots.	No ambiguity among type, usage, instance, owner, context, or presentation.	Do not combine evidence approval or multi-site rollup.

3. Requirement and allocation architecture

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Implement complete requirement fields, relations, applicability, and matrices.	Stages 1–2.	Requirement services, trace semantics, suspect foundation, workbook schema.	Relationship direction and applicability tests pass; incomplete claims blocked.	Do not treat verify links as status.

4. Unit verification-scope engine

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Generate deterministic, explainable categorized scope.	Stages 1–3 and interface graph baseline.	Scope rule model, evaluator, explanation graph, workspace, reports.	AC-01 through AC-04 pass with positive/negative fixtures.	Do not add execution management before scope semantics stabilize.

5. Test planning, execution, and evidence

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Separate Test Case, planned use, execution, results, evidence, anomaly, and approval.	Stages 1–4; identity/configuration services.	Verification metamodel, plan/execution workbenches, evidence references, reports.	AC-05, AC-09, AC-15 pass; links alone never grant status.	Do not implement automatic cross-configuration reuse.

6. Interface-aware impact analysis

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Compute interface/dependency scope and precise regression effects.	Validated ports/connectors/ flows; stages 3–5.	Traversal rules, adequacy assessment, impact engine, suspect/regression views.	AC-03, AC-07, AC-08 pass without mass invalidation.	Do not hide unresolved interface semantics with heuristics.

7. Configuration and baseline management

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Control as-designed/built/installed/tested/maintained identity and evidence applicability.	Stages 2 and 5–6.	Baseline lifecycle, comparisons, reuse assessments, approval rules.	AC-05 and AC-11 pass across changes and reload.	Do not collapse revision, baseline, and configuration.

8. Multi-site verification rollup

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Reuse definitions while preserving local installations/evidence and transparent rollup.	Stages 2–7.	Site architecture, shared library governance, comparison, program dashboard.	AC-10 and coverage rollup tests pass with divergent sites.	Do not duplicate common definitions or average incompatible denominators.

9. Performance and scalability qualification

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Meet defined capacity, latency, reliability, and recovery targets.	Stable functional semantics from stages 1–8.	Benchmarks, profiling, indexing/caching, large-model fixtures, recovery tests.	Approved performance targets pass without semantic shortcuts.	Do not change semantics solely to improve benchmarks.

10. Production governance and interchange qualification

Objective	Dependencies	Deliverables	Acceptance gate	Must not be combined
Qualify roles, approvals, audit, retention, collaboration, security, and interchange.	All earlier stages and deployment architecture decisions.	Permission model, tamper-evidence, retention, operational controls, qualified import/export, collaboration tests.	AC-13–15 and production-readiness review pass.	Do not claim vendor or standard certification without independent evidence.

Architecture Decisions

Decision	State	Consequence
ADR-001 Semantic repository is authoritative	Accepted	All diagrams, tables, matrices, dashboards, and reports are derived views.
ADR-002 Semantic and presentation identities are separate	Accepted	One semantic element may have many view-specific presentations.
ADR-003 Diagram owner and semantic context are separate	Accepted	Every diagram has explicit package ownership and compatible context.
ADR-004 Selective child diagrams	Accepted	Create/link child diagrams only when decomposition is modeled.
ADR-005 Type/usage/instance/configuration separation	Accepted	Evidence and scope bind to installed identity and controlled configuration.
ADR-006 Explainable rule-based scope	Accepted	Every inclusion/exclusion stores rule, path, baseline, and reason.
ADR-007 Evidence-derived verification status	Accepted	Links and manual toggles cannot alone grant verified status.
ADR-008 Transactional identity-preserving import	Accepted	Dry run, reconciliation, rollback, and no silent orphans are mandatory.
ADR-009 SysML 1.x baseline	Accepted	No SysML 2.0 or vendor-certification claim without separate implementation and qualification.
ADR-010 Collaboration uses semantic operations	Accepted direction	Future collaboration must synchronize fine-grained semantic/presentation operations, not replace whole projects.

Unresolved Decisions

Decision	Required resolution	Why it matters
UD-01 Canonical repository technology	Select local-first database/event log and server-backed production authority while preserving offline browser use.	Identity, transactions, collaboration, scalability.
UD-02 Rule language	Choose declarative representation/versioning for applicability, scope, rollup, and impact rules.	Explainability and qualification.
UD-03 Evidence storage	Choose embedded, linked, or managed object-store strategy; define hashing, access, retention, and export.	Security, file size, portability.
UD-04 Electronic approvals	Define identity assurance, signature meaning, revocation, and regulatory needs.	Verification authority and audit.
UD-05 Baseline immutability	Define immutable snapshots versus controlled branches/tags and supersession.	Reproducibility and storage.
UD-06 Collaboration conflict model	Choose operation log/CRDT/OT or transaction-based approach for semantic and presentation edits.	Convergence and user conflict handling.
UD-07 Performance targets	Set model sizes, concurrent users, import volume, scope latency, and report-generation targets.	Stage 9 gates.
UD-08 SysML profile boundary	Approve exact metaclasses, stereotypes, notation variants, and domain extensions.	Professional conformance matrix.

Decision	Required resolution	Why it matters
UD-09 Proprietary interchange boundary	Decide supported neutral/structured exchanges without claiming native CATIA/3DEXPERIENCE compatibility.	User expectations and test fixtures.
UD-10 Qualification level	Define intended internal assurance and any future regulatory/contractual qualification.	Depth of audit, validation, and approvals.

Future Codex Implementation Prompts

Prompt ID	Required implementation prompt scope
P-01 Baseline qualification	Audit the current repository against the professional SysML element/diagram matrix; fix only verified regressions; add save/reload and notation tests.
P-02 Semantic/presentation split	Implement stable semantic IDs and separate presentation IDs, explicit diagram owner/context, multi-diagram reuse, and safe presentation deletion.
P-03 Type-instance configuration	Implement reusable definitions, contextual usages, serial instances, variants, revisions, and configuration snapshots with migration tests.
P-04 Requirement architecture	Implement complete requirement fields, directional relationship semantics, applicability, validation, and matrices.
P-05 Scope rule model	Define versioned rule schema and explanation graph for direct, interface, dependency, regression, higher-level, and exclusion categories.
P-06 Scope engine UI	Build the Unit Verification Workspace with filters, deterministic evaluation, coverage formula, explanations, exclusions, and unresolved-problem warnings.
P-07 Verification metamodel	Implement Test Case, Planned Test, procedure/steps, configuration, execution, result, measurement, anomaly, evidence, and approval as distinct entities.
P-08 Interface adequacy	Implement operational-to-test interface mapping, simulator/fixture adequacy assessment, and evidence validity rules.
P-09 Impact and suspect links	Implement baseline diff, relationship-specific propagation, precise unaffected results, regression plans, and explanations.
P-10 Import v2	Implement dry-run, identity-preserving transactional import/reimport for modeling and verification workbooks with rollback and reconciliation reports.
P-11 Multi-site rollup	Implement shared library definitions, site-installed occurrences, local evidence, transparent rollup, and configuration-aware cross-site comparison.
P-12 Governance and collaboration	Implement roles, approvals, audit, retention, semantic operation synchronization, conflict handling, and collaboration consistency tests.
P-13 Performance qualification	Create large deterministic fixtures, capacity targets, profiling, indexes, recovery tests, and semantic-equivalence assertions.
P-14 Report qualification	Generate reproducible scope, impact, and unit verification reports tied to baseline, rule set, configuration, evidence, and approvals.

Every future Codex prompt must reference the relevant acceptance criteria, preserve unrelated working behavior, require automated tests, state migrations, and prohibit unsupported conformance claims.

Glossary

Term	Definition
Applicability	Rule determining whether a requirement, test, or evidence claim pertains to a site, type, usage, instance, variant, configuration, baseline, or condition.
Baseline	Approved identified set of model versions and relationships used for reproducibility.
Configuration	Identified selection and state of items, revisions, variants, connections, software, settings, and conditions.
Direct scope	Requirements attributable to and verifiable at the selected level.
Evidence	Controlled artifact or data supporting a verification result and conclusion.
Execution	A specific historical occurrence of a verification activity.
Explanation graph	Machine-readable rules and relationship paths justifying scope or impact results.
Installed usage	Contextual occurrence of a reusable type in a containing system/site structure.
Interface scope	Requirements necessary to verify interactions across the selected boundary.
Planned Test	Plan-specific intended use of a reusable Test Case.
Presentation element	View-specific node, edge, label, compartment, or geometry referring to semantic identity.
Requirement verification status	Governed conclusion based on applicable, admissible evidence and approval.
Semantic element	Authoritative repository object independent of any particular view.
Scope Evaluation	Reproducible result of applying a versioned rule set to selected inputs and a baseline.
Suspect	State indicating that a change or missing fact may invalidate a relationship, result, reuse decision, or conclusion.
Test Case	Reusable verification definition specifying objective, conditions, inputs, and expected outcome.
Test Configuration	Controlled identity of article, interfaces, equipment, software, settings, and environment used for an execution.
Verification credit reuse	Explicit approved use of source evidence for a different target based on controlled equivalence.
Verification method	Test, analysis, inspection, demonstration, simulation, certification, or justified similarity/equivalence.
Verification workbench	Repository-derived workspace for scope, planning, execution, evidence, status, impact, and reporting.

Specification Completion Checklist

- Product purpose and lowest-valid-level verification principle are normative.
- All six scope categories have inclusion, exclusion, trace, explanation, warning, reporting, and example rules.
- Ownership, decomposition, allocation, scope, and presentation are distinct.
- Type, instance, configuration, baseline, site identity, and evidence applicability are mandatory.

- Requirements, verification entities, methods, scope engine, impact, interfaces, behavior, parametrics, multi-site use, views, governance, import, and SysML scope are defined.
- Acceptance criteria provide preconditions, actions, semantic/UI results, persistence, automated tests, and failures.
- Roadmap stages identify objectives, dependencies, deliverables, gates, and prohibited combinations.
- Architecture decisions, unresolved decisions, glossary, and future Codex prompts are included.